



Реализация REST API на основе контракта

Цель работы

На предыдущем занятии мы спроектировали и создали самодостаточный, строго типизированный модуль-контракт `book-library-api`. Мы определили «что» делает наш сервис. Теперь пришло время создать сервис, который будет реализовывать этот контракт — определить, «как» он это делает. Мы создадим отдельный микросервис, подключим к нему наш контракт как зависимость и напомним бизнес-логику.

Вводные

Нам нужно:

Проект с контрактом (у нас `book-library-api`), созданный на предыдущем занятии.

Соберите для него `jar` файл, со всеми зависимостями — гайд тут: <https://www.jetbrains.com/guide/java/tutorials/hello-world/packaging-the-application/>

Выполняем по гайду, далее в Idea – Build – Build Artifacts, будет папка `out`, в ней и найдете `jar`.

Проект для сервиса, который реализует данный контракт. Создайте проект на <https://start.spring.io/> и добавьте Spring Web в качестве зависимости, остальное придет транзитивно от проекта с контрактом (на всякий можно и добавить отдельно).

Открыв проект, подключите `Jar` как зависимость:

Создайте директорию `lib`

В нее скопируйте `jar` файл

В `pom`:

```
<dependency>
  <groupId>edu.rut-miit.demo</groupId>
  <artifactId>demo_web_1</artifactId>
  <version>1.0</version>
  <scope>system</scope>

  <systemPath>${project.basedir}/lib/demo_web_1.jar</systemPath>
</dependency>
```

Где `demo_web_1` название вашего `Jar`

Не забывайте нажимать обновить зависимости!

Слой контроллера (@RestController)



СОП

Это класс в Spring, отвечающий за обработку входящих HTTP-запросов. Его основная задача — принять запрос, извлечь из него данные (параметры пути, тело запроса) и передать их на обработку в сервисный слой. Контроллер должен быть "тонким", то есть не содержать сложной бизнес-логики.

Подключив контракты, реализуйте интерфейсы:

```
@RestController
public class BookController implements BookApi {
    // ... реализация методов интерфейса ...
}
```

Ключевой момент здесь — `implements BookApi`. Это заставляет нас реализовать все методы, объявленные в контракте, и автоматически наследовать все OpenAPI и Web-аннотации из интерфейса.

Слой сервиса (@Service)

Класс, содержащий основную бизнес-логику приложения. Здесь происходят вычисления, проверки, взаимодействие с базой данных и другая "работа". Сервисный слой ничего не знает про HTTP, его задача — выполнить бизнес-операцию. Такой подход называется "разделением ответственности" (Separation of Concerns).

```
@Service
public class BookService {
    public BookResponse findBookById(Long id) {
        // ... логика поиска книги ... можно просто в массиве
    }
}
```

Внедрение зависимостей

Механизм, с помощью которого Spring Framework автоматически управляет зависимостями между компонентами. Вместо того чтобы создавать объект сервиса внутри контроллера (`BookService service = new BookService();`), мы просим Spring "внедрить" его нам.

Внедрим через конструктор сервис:

```
@RestController
public class BookController implements BookApi {

    private final BookService bookService;
```



СОП

```
// Spring автоматически найдет бин типа BookService и
// передаст его в конструктор
@Autowired
public BookController(BookService bookService) {
    this.bookService = bookService;
}

// ...
}
```

Пример готового контроллера:

```
@RestController
public class BookController implements BookApi {

    private final BookService bookService;

    @Autowired
    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @Override
    public BookResponse getBook(Long id) {
        return bookService.findBookById(id);
    }

    @Override
    public BookResponse createBook(@Valid BookRequest request) {
        BookResponse createdBook =
        bookService.createBook(request);
        return createdBook;
    }
}
```

Основное преимущество в том, что все наши аннотации будут работать транзитивно, нет необходимости в дублировании.

Для точного успеха дублирую роты сервисов:

Контракт:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```



СОП

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.8.13</version>
</dependency>

<dependency>
  <groupId>jakarta.validation</groupId>
  <artifactId>jakarta.validation-api</artifactId>
  <version>3.1.1</version>
</dependency>
</dependencies>
```

Сервис:

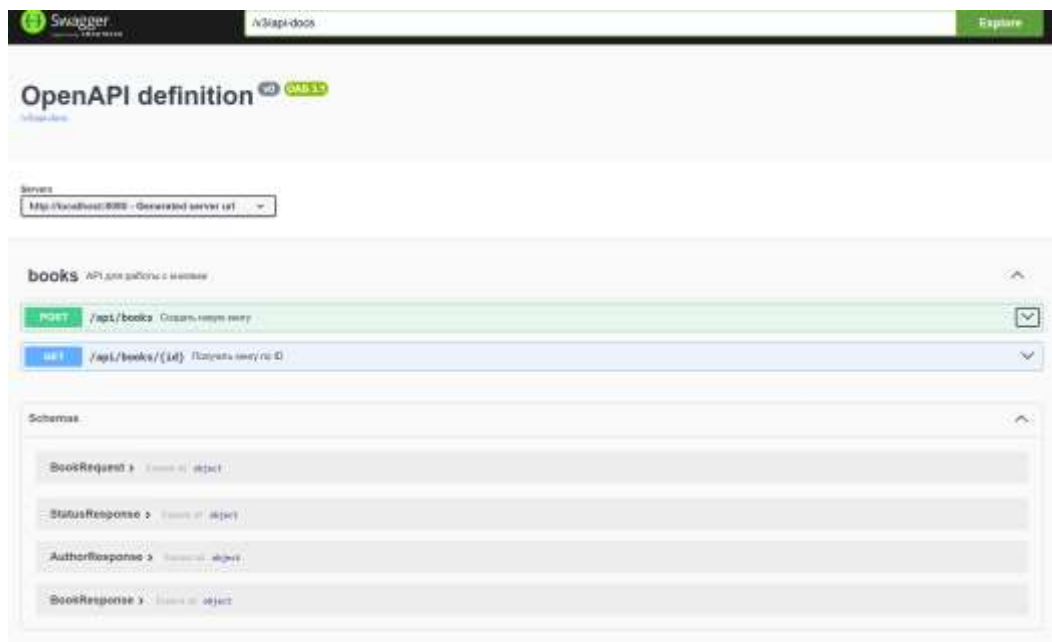
```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.8.13</version>
  </dependency>
  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-api</artifactId>
    <version>2.8.13</version>
  </dependency>
  <dependency>
    <groupId>jakarta.validation</groupId>
    <artifactId>jakarta.validation-api</artifactId>
    <version>3.1.1</version>
  </dependency>
  <dependency>
    <groupId>edu.rut-miit.demo</groupId>
    <artifactId>books-api</artifactId>
    <version>1.4</version>
    <scope>system</scope>
    <systemPath>${project.basedir}/lib/books-api-
contract.jar</systemPath>
  </dependency>
</dependencies>
```

Проекты прилагаются.

При запуске приложения идем в <http://localhost:8080/swagger-ui/index.html> и любуемся swagger'ом.



СОП



Задания для самостоятельной Работы

Подключите Jar вашего проекта контрактов к реализации в отдельный проект сервиса и реализуйте все интерфейсы. Сделайте сервисы, которые работают с данными в оперативной памяти (массивы объектов), протестируйте работу сервиса.